

TICKET BOOKING SYSTEM

Production-grade high-concurrency roadmap

Two weeks | 12 working days | 35 focused hours

System design -> zero-oversell engine -> reliability -> scale -> observability -> CI/CD -> deployment

Primary engineering goal

Build a system whose correctness can be demonstrated under contention. A flashy architecture is irrelevant if 101 tickets can be sold from an inventory of 100. The project should produce measurable evidence: zero oversells, deterministic idempotency, safe expiration, stable multi-instance behavior, and load-test results.

Recommended stack

Layer	Choice	Why
Runtime	Java 21 + Spring Boot 4.x	Modern JVM, strong ecosystem, production CV signal
Database	PostgreSQL + Flyway	Authoritative inventory, transactions, row locking, indexes
Cache	Redis	Catalog cache, rate limiting, optional hot reads - never stock authority
Messaging	RabbitMQ	Reliable async jobs/events without over-engineering Kafka
Testing	JUnit 5, Testcontainers, k6	Correctness + real PostgreSQL concurrency + load proof
Observability	Actuator, Micrometer, Prometheus, Grafana	Metrics, health, dashboards, alerts
Delivery	Docker, Compose, GitHub Actions, GHCR, Nginx/load balancer	CI gates, immutable images, automated deploy + rollback

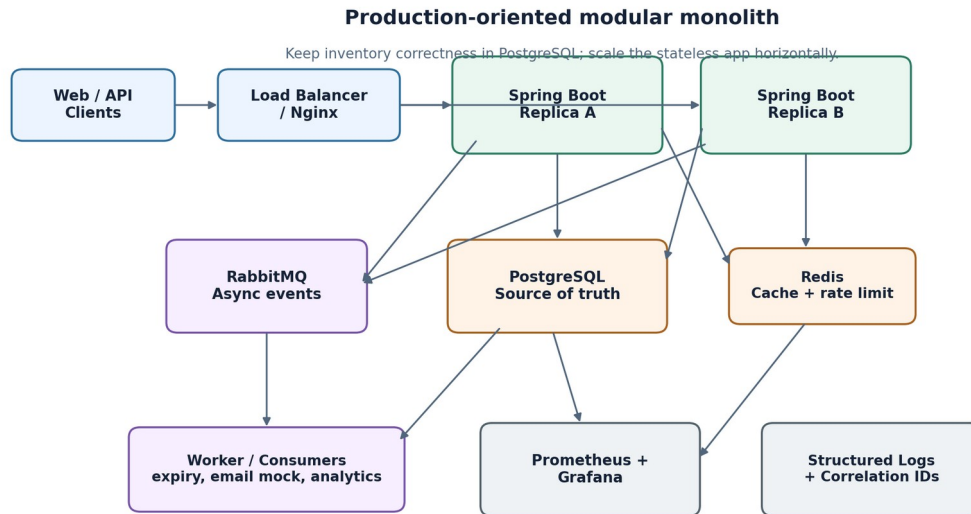
North-star acceptance targets

- 0 oversold tickets in every concurrency test.
- Duplicate client retries create exactly one reservation.
- Expired or cancelled holds return inventory exactly once.
- Two application replicas preserve correctness under concurrent traffic.
- Target p95 reserve latency ≤ 300 ms at 500 concurrent virtual users on the chosen environment, or document the bottleneck with evidence.
- Metrics and logs explain failures without attaching a debugger.

[START HERE](#)

1. Architecture and Engineering Strategy

Use a modular monolith first. In a two-week build, microservices add network failure, distributed transactions, deployment overhead, and debugging cost without improving the core proof: safe concurrent booking.



Core modules

Module	Responsibility
catalog	Events, venues, show times, sale windows
inventory	Authoritative ticket counts and lock/atomic update path
reservation	Hold lifecycle, TTL, user limits, idempotency
order/payment	Confirmation flow; payment can be a deterministic mock
messaging	Outbox publisher, RabbitMQ consumers, DLQ
security	JWT/RBAC, validation, rate limiting
observability	Metrics, tracing/log correlation, health endpoints

Hard boundary

PostgreSQL is the source of truth for inventory. Redis can accelerate reads but a Redis cache miss, restart, or stale value must never create or lose tickets.

Do not build these in the first 35 hours

- Microservices/Kubernetes before the monolith is proven under load.
- Kafka or event sourcing without a measured replay/streaming requirement.
- Real payments; use a deterministic fake gateway to test retries and failures.
- Redis as inventory authority; PostgreSQL remains the commit-time truth.

BEFORE CODING

2. Non-Negotiable Correctness Model

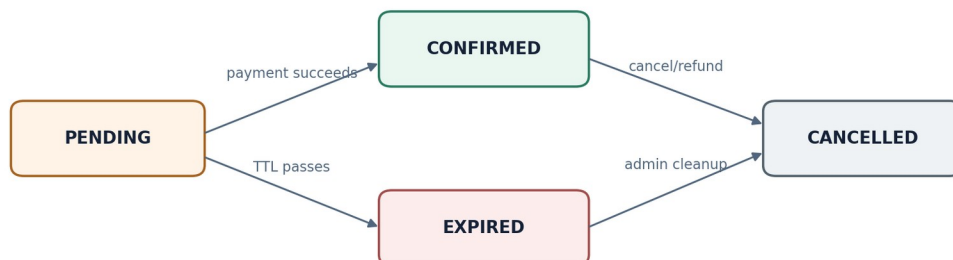
Business invariants

ID	Invariant	Meaning
I1	available_tickets >= 0	No overselling under any interleaving.
I2	available + held + sold = total	Inventory conservation.
I3	One idempotency key -> one logical reservation	Safe client retries.
I4	A hold is released at most once	Expiry/cancel workers are idempotent.
I5	Confirmation consumes the same hold exactly once	No double-selling through payment retries.
I6	Per-user/event purchase cap enforced inside transaction	No race bypass of business limits.

Minimal relational model

Table	Important fields	Purpose
event	id, name, venue, sale_starts_at, status	Catalog / sale window
ticket_inventory	event_id PK/FK, total, available, held, sold, version/update time	Single hot row per event for MVP
reservation	id UUID, event_id, user_id, qty, status, expires_at, idempotency_key UNIQUE	Hold state + retry identity
order	id, reservation_id UNIQUE, status, amount, created_at	Confirmation/payment boundary
outbox_event	id, aggregate_id, type, payload, published_at, created_at	Reliable event publication

Reservation state machine



Critical transaction - first implementation

Inside one database transaction: lock the inventory row with `SELECT ... FOR UPDATE`, re-check sale/user limits, validate `available >= quantity`, decrement available and increment held, insert PENDING reservation, commit. Do not rely on an earlier availability read from Redis or an unlocked query.

17.5 HOURS

3. Week 1 - Correctness First

DAY 1 | 3.0 hours

System design, SLOs, and failure model

Objective: Define exactly what the system guarantees before implementation.

Build

- Draw the request path and module boundaries; choose modular monolith + stateless app replicas.
- Write ADR-001 for PostgreSQL as inventory authority and ADR-002 for pessimistic locking on the initial reserve path.
- Define SLO-style project targets: zero oversells, p95 latency target, acceptable error rate, recovery expectations.
- List failure cases: duplicate POST, client timeout after commit, app crash, DB restart, broker outage, two replicas racing.
- Define APIs: POST /events/{id}/reservations, POST /reservations/{id}/confirm, DELETE /reservations/{id}.

Deliverable: architecture.md + ADRs + API contract + failure matrix.

Acceptance gate: You can explain how every listed failure is contained before writing the implementation.

Do not waste time on: Drawing ten services. Keep boundaries logical inside one deployable application.

DAY 2 | 3.0 hours

Schema, migrations, indexes, and state machine

Objective: Make illegal states difficult to represent and cheap to query.

Build

- Create Flyway migrations for event, ticket_inventory, reservation, order, outbox_event.
- Add database constraints: qty > 0, counters >= 0, unique idempotency key, one order per reservation.
- Add indexes for event/status/expires_at and user/event active reservation lookups.
- Map entities carefully; avoid cascading large object graphs and accidental N+1 queries.
- Implement ReservationStatus transitions with service methods rather than exposing status setters.

Deliverable: Repeatable schema + seed migration for a hot event with exactly 100 tickets.

Acceptance gate: Database can be rebuilt from zero; invalid counters/duplicate idempotency keys are rejected by constraints.

Do not waste time on: Building admin UI or complex seat maps. General-admission inventory is enough to prove concurrency.

DAYS 3-4

Week 1 - Critical Reservation Path

DAY 3 | 3.25 hours

Zero-oversell reservation transaction

Objective: Make the race condition correct before making it fast.

Build

- Implement repository query using PESSIMISTIC_WRITE / SELECT FOR UPDATE for the event inventory row.
- Inside @Transactional: validate sale window, quantity, per-user cap, and available inventory after the lock is acquired.
- Decrement available, increment held, create PENDING reservation with short TTL (for example 2-5 minutes).
- Return clear domain errors: SOLD_OUT, SALE_NOT_STARTED, USER_LIMIT_EXCEEDED, INVALID_QUANTITY.
- Add a multithreaded integration test using real PostgreSQL via Testcontainers and a barrier so requests start together.

Deliverable: Working reserve endpoint + deterministic concurrent integration test.

Acceptance gate: 100 tickets + hundreds of competing reserve attempts -> accepted quantity never exceeds 100; database counters reconcile exactly.

Do not waste time on: Performance tuning before you have a failing contention test.

DAY 4 | 3.0 hours

Idempotency, expiration, cancellation

Objective: Make retries and abandoned carts safe.

Build

- Require Idempotency-Key on reservation creation; enforce uniqueness in PostgreSQL, not only in Java memory.
- If the same key is retried, return the original logical result rather than inserting a second hold.
- Implement cancellation transaction: lock reservation/inventory, transition once, return held quantity once.
- Implement expiration worker in batches. Use row locks and SKIP LOCKED so multiple workers can safely cooperate.
- Make expiry and cancellation operations idempotent; repeated calls must be no-ops after the first valid transition.

Deliverable: Retry-safe API + expiration worker + cancellation path.

Acceptance gate: 10 identical retries -> 1 hold. Two expiry workers -> inventory returned once. Cancel after expiry -> no double increment.

Do not waste time on: Using a single in-memory scheduler lock; it breaks as soon as you run two app replicas.

DAYS 5-6

Week 1 - Guardrails and Proof

DAY 5 | 2.5 hours

Security, validation, and Redis fast paths

Objective: Protect the critical path without moving correctness out of the database.

Build

- Finish JWT authentication and role checks for user/admin endpoints if not already present.
- Validate request sizes and quantities at the API boundary; use consistent error response format.
- Add Redis cache for event/catalog reads with short TTL and explicit cache invalidation on admin updates.
- Add per-user/IP rate limiting for reserve attempts; keep rate-limit failure separate from SOLD_OUT.
- Add correlation/request IDs to logs and propagate them through services.

Deliverable: Protected API + catalog cache + rate limit + structured error contract.

Acceptance gate: Redis unavailable -> booking correctness still works; catalog can fall back to PostgreSQL or fail clearly without overselling.

Do not waste time on: Caching available ticket count as an authority. Stale availability is acceptable for display, never for commit.

DAY 6 | 2.75 hours

k6 baseline and contention analysis

Objective: Turn correctness claims into numbers and identify the actual bottleneck.

Build

- Create k6 scenarios: 100 tickets / 500 VUs, duplicate retries, mixed reserve/cancel, and sold-out spike.
- Record accepted reservations, errors by domain code, p50/p95/p99 latency, throughput, and DB connection usage.
- Assert server-side reconciliation after each run: available + held + sold = total and no negative counters.
- Inspect PostgreSQL lock waits and Hikari pool saturation; do not guess which component is slow.
- Commit load scripts and a results.md with machine/environment details so results are reproducible.

Deliverable: Baseline performance report with graphs/screenshots and zero-oversell evidence.

Acceptance gate: 500-VU scenario finishes with exactly valid inventory. If p95 misses target, the report identifies the measured reason.

Do not waste time on: Inventing impressive RPS numbers. Reproducibility is more valuable on a CV than a fake benchmark.

17.5 HOURS

4. Week 2 - Reliability and Scale

DAY 7 | 3.0 hours

Reliable async events with Outbox + RabbitMQ

Objective: Decouple non-critical work without losing events at transaction boundaries.

Build

- Write outbox_event in the same transaction that changes reservation/order state.
- Create publisher worker that sends unpublished rows to RabbitMQ and marks them published only after broker acknowledgement.
- Add queues for reservation-confirmed, notification mock, and analytics/audit work.
- Make consumers idempotent using event IDs / processed-event table or natural unique keys.
- Configure retry with bounded exponential backoff and a dead-letter queue for poison messages.

Deliverable: Outbox publisher + RabbitMQ topology + one idempotent consumer + DLQ.

Acceptance gate: Kill broker during confirmation, restore it, and verify the event is eventually delivered once logically without corrupting booking state.

Do not waste time on: Publishing directly to RabbitMQ inside the booking transaction and assuming the DB + broker commit atomically.

DAY 8 | 2.75 hours

Failure handling and recovery behavior

Objective: Make dependencies fail predictably instead of hanging the system.

Build

- Set explicit DB/broker/http timeouts; avoid unbounded waits.
- Use retries only for safe/idempotent operations; never blind-retry a non-idempotent purchase action.
- Add readiness/liveness health indicators; readiness should fail when the app cannot safely serve traffic.
- Handle graceful shutdown so in-flight requests finish and consumers stop cleanly.
- Run failure drills: app kill during load, RabbitMQ outage, Redis outage, slow database, duplicate delivery.

Deliverable: Failure-mode test notes + configured timeouts/retries + health probes.

Acceptance gate: Every failure drill has an expected result and the system recovers without lost/duplicated inventory.

Do not waste time on: Adding circuit breakers everywhere. Use them only on remote calls where they prevent cascading failure.

DAYS 9-10

Week 2 - Scaling and Observability

DAY 9 | 3.0 hours

Horizontal scale and hot-row optimization

Objective: Prove that multiple app instances stay correct; optimize only the measured contention point.

Build

- Run two Spring Boot replicas behind Nginx/load balancer; ensure sessions are stateless.
- Tune Hikari pool conservatively relative to PostgreSQL max connections; more connections are not automatically faster.
- Benchmark current pessimistic-lock path at 500 -> 1,000 -> 2,000 VUs as environment allows.
- If lock wait dominates, implement and benchmark conditional atomic inventory decrement as an alternative branch.
- Cache read-heavy event details; add pagination and query projection for catalog endpoints; verify indexes with EXPLAIN ANALYZE.

Deliverable: Multi-instance deployment locally + before/after benchmark for the one chosen optimization.

Acceptance gate: Two replicas, one database, heavy contention -> still zero oversells. Keep whichever transaction strategy wins on measured latency/throughput while preserving invariants.

Do not waste time on: Optimistic locking on a deliberately hot inventory row, or sharding before a single-node database is proven insufficient.

DAY 10 | 2.75 hours

Metrics, dashboards, logs, and alerts

Objective: Make system behavior explainable in production.

Build

- Enable Actuator + Micrometer Prometheus endpoint.
- Add business metrics: reservation attempts/success/sold-out, confirmation success/failure, expired holds, idempotency hits.
- Add latency timers around reserve transaction and expiry worker; expose JVM, Hikari, HTTP, and RabbitMQ metrics.
- Create Grafana dashboard: traffic, p95/p99, error rate, DB pool, JVM, inventory outcomes, queue depth.
- Add alert rules for error-rate spike, high p95, DB pool saturation, queue backlog, and failed health checks.

Deliverable: One Grafana dashboard + alert rules + structured logs with correlation IDs.

Acceptance gate: You can diagnose a failed load test from dashboard + logs without opening IntelliJ.

Do not waste time on: Distributed tracing infrastructure if there is only one deployable service. Metrics + correlated logs give more value first.

DAYS 11-12

Week 2 - Verification and Deployment

DAY 11 | 3.0 hours

Production test suite and release hardening

Objective: Prove state correctness across races, restarts, and duplicates.

Build

- Unit-test domain transition rules and validation logic.
- Use Testcontainers for repository/transaction tests against PostgreSQL and RabbitMQ where useful.
- Add concurrency suites for reserve, cancel, confirm, expiry, duplicate idempotency key, and two-worker expiry.
- Add API integration tests for auth, validation, error contract, and idempotent responses.
- Run a soak test (30-60 minutes if possible) and verify no memory/connection leak; record final metrics.

Deliverable: Release-candidate test report with pass/fail matrix and known limits.

Acceptance gate: All invariants hold across the full test matrix; failures are reproducible and documented instead of hidden.

Do not waste time on: Chasing 100% line coverage. Cover risky state transitions and concurrency behavior first.

DAY 12 | 3.0 hours

Containerize, deploy, prove, and package for CV

Objective: Finish with a reproducible deployed system and evidence, not just source code.

Build

- Create optimized Dockerfile and docker-compose.yml for local dependencies; add healthchecks.
- Run Flyway automatically/safely on deployment; move secrets to environment/secret store and remove them from Git history.
- Deploy at least two stateless app replicas behind HTTPS/load balancer when the platform allows; use managed PostgreSQL/Redis/RabbitMQ or equivalent services.
- Run smoke tests and the 500-VU correctness scenario against the deployed environment within provider limits.
- Finish README: architecture, invariants, tradeoffs, run commands, load-test results, dashboard screenshots, failure drills, known limitations.

Deliverable: Live deployment + tagged v1.0.0 + final benchmark evidence + polished README.

Acceptance gate: A reviewer can clone, run, understand the concurrency strategy, and see evidence of zero overselling and observability in under 10 minutes.

Do not waste time on: Adding new features on the last day. Freeze scope and polish proof, docs, and deployment.

RELEASE AUTOMATION

CI/CD - build once, deploy safely

Day 12 implementation detail | Included inside the existing 3.0-hour timebox

Goal

A commit is not a release. Every production version must pass the same automated gates, produce an immutable image, deploy predictably, and prove health after release.

Pipeline



GitHub Actions workflow

Trigger	Pull requests run CI. Merge to main produces a releasable image. Version tags such as v1.0.0 create formal releases.
Quality gates	Run Maven verify, unit/integration tests, Testcontainers concurrency tests, and fail the pipeline on any invariant violation.
Security	Run dependency checks and scan the built container image (for example, Trivy). Do not ship a known critical vulnerability.
Artifact	Build the Docker image once. Tag it with the Git commit SHA; never rebuild a different image on the production server.

Safe production release

Registry	Push immutable images to GitHub Container Registry (GHCR). Keep the previous successful SHA available for rollback.
Secrets	Use GitHub Environments/Secrets or cloud OIDC. Database credentials, JWT secrets, and broker passwords never enter the repository or image.
Deploy	Use the hosting platform's rolling deployment when available. On a VM, deploy a new app instance/stack, verify readiness, then switch Nginx traffic.
Verify	After deploy, call readiness/health endpoints and run a small smoke reservation flow. A failed check must stop the release.
Rollback	Retain the previous image tag and automate restoration when post-deploy health checks fail. Database migrations must remain backward compatible for the release window.

Day 12 - revised 3-hour timebox

45 min	CI workflow: Maven verify + integration/concurrency tests
30 min	Docker build, immutable SHA tag, image/security scan, push to GHCR
60 min	Automated deployment + readiness gate + rollback path
45 min	Production smoke test, release evidence, README and v1.0.0 tag

Acceptance gate

- A failing test cannot produce or deploy a release image.
- Merge/tag produces an immutable image and deploys it through automation - no manual server build.
- Failed readiness/smoke check prevents traffic cutover or restores the last known-good image.
- The README contains the pipeline diagram, deployment method, and one successful release run screenshot.

EVIDENCE

5. Test Matrix - What Makes the Project Credible

Scenario	How to trigger	Pass condition
Inventory race	100 tickets; 500 concurrent 1-ticket attempts	Exactly 100 tickets held/sold; available never negative.
Burst sold-out	Inventory reaches zero while traffic continues	Remaining requests get SOLD_OUT quickly; no 5xx storm.
Duplicate request	Same Idempotency-Key sent 10-50 times	One reservation, stable response semantics.
Per-user limit race	Same user sends parallel reservations	Total active quantity never exceeds configured cap.
Expiry race	Two workers process same expired batch	Each hold released exactly once.
Cancel vs expire	Cancellation and expiry race	One valid transition; inventory returned once.
Confirm retry	Same confirmation delivered/retried	One order/confirmation; no double sold increment.
Broker outage	RabbitMQ unavailable during state change	Booking commits; outbox retains event; publishes after recovery.
Redis outage	Cache/rate-limit dependency unavailable	Authoritative booking logic cannot oversell.
Multi-instance	2+ app replicas under same hot event	Correctness identical to single instance.
App crash	Kill one replica during load	Other replica serves; committed DB state remains valid.
Soak	30-60 min mixed traffic	Stable memory, thread, connection, and queue behavior.

Suggested k6 progression

- Gate 1 - 100 VUs: prove scripts and error semantics.
- Gate 2 - 500 VUs: primary CV benchmark and zero-oversell demonstration.
- Gate 3 - 1,000 VUs: measure lock wait, DB pool, CPU, and p95 degradation.
- Gate 4 - 2,000+ VUs only if your environment can drive it honestly; document client-side limits as well as server-side limits.

Benchmark rule

Never report VUs/RPS without the hardware or hosting tier, test duration, ticket count, p95/p99, error mix, and post-test inventory reconciliation. The number is meaningless without context.

BEFORE V1.0

6. Production Readiness Checklist

Correctness

- ☐ All inventory mutations are transactional.
- ☐ Idempotency key is database-enforced.
- ☐ Reservation transitions are explicit and one-way where required.
- ☐ Expiry/cancel/confirm are safe under retries and races.
- ☐ Post-test reconciliation query proves inventory conservation.

Reliability

- ☐ Outbox prevents DB/broker dual-write loss.
- ☐ Consumers are idempotent; DLQ is configured.
- ☐ Timeouts are explicit; retries are bounded.
- ☐ Graceful shutdown and health probes are verified.
- ☐ Backups/restore procedure for production PostgreSQL is documented.

Scalability

- ☐ Application is stateless and runs with two replicas.
- ☐ DB pool size is intentional.
- ☐ Hot queries have indexes and EXPLAIN evidence.
- ☐ Redis is used for read acceleration/rate limits, not inventory truth.
- ☐ Measured contention decides whether pessimistic lock or atomic update is kept.

Observability

- ☐ Prometheus endpoint is protected appropriately.
- ☐ Dashboard shows throughput, p95/p99, error rate, DB pool, JVM, queue depth, business outcomes.
- ☐ Logs are structured and carry correlation IDs.
- ☐ Alerts exist for high errors, latency, pool saturation, queue backlog, failed readiness.

Delivery

- ☐ Docker image uses a minimal runtime image and non-root user where practical.
- ☐ Secrets are injected, not committed.
- ☐ Flyway migration strategy is documented.
- ☐ HTTPS and reverse proxy/load balancer are configured.
- ☐ README contains one-command local startup and reproducible load tests.

FINAL PACKAGING

7. CV Impact - Convert Engineering Work Into Evidence

What recruiters should learn in 30 seconds

- You understand concurrency beyond synchronized blocks: transactions, row locks, idempotency, and failure races.
- You can measure systems: k6, p95/p99, saturation, database contention, dashboards.
- You design reliable async workflows: transactional outbox, acknowledgements, retries, DLQ, idempotent consumers.
- You can ship: migrations, containers, secrets, health checks, reverse proxy/load balancing, deployment.
- You make tradeoffs: modular monolith before microservices; database correctness before cache cleverness.

Example CV bullets - replace placeholders with measured numbers

- Built a Java/Spring Boot ticket-booking backend that preserved zero overselling under [500+] concurrent virtual users using PostgreSQL transactional concurrency control and database-enforced idempotency.
- Designed reservation expiry/cancellation and confirmation state transitions to remain correct under retries and multi-worker races; validated with Testcontainers concurrency tests and k6 load scenarios.
- Implemented a transactional outbox with RabbitMQ, idempotent consumers, retry/backoff, and DLQ to guarantee reliable asynchronous processing across broker outages.
- Deployed stateless application replicas behind a load balancer with Redis caching/rate limiting, Prometheus/Grafana observability, structured logs, health probes, and reproducible Docker-based environments.
- Profiled hot-row contention and database connection saturation, then benchmarked [pessimistic locking vs conditional atomic SQL] to improve p95 reservation latency from [X] ms to [Y] ms without weakening inventory invariants.

README evidence package

- Architecture diagram and ADRs.
- Data model + reservation state machine.
- Concurrency strategy with why it is correct.
- k6 commands + exact test parameters + results table.
- Screenshots of Grafana under load.
- Failure-drill notes: broker outage, Redis outage, app kill, duplicate requests.
- Known limits and next scale step - e.g., split inventory by section/seat bucket, dedicated inventory service only when justified.

Definition of done

The project is finished when the deployed system can be attacked with concurrent requests, still reconcile inventory exactly, recover from dependency failures, expose enough telemetry to explain its behavior, and present reproducible evidence in the repository. A long feature list without those proofs is weaker.

Stretch goals only after v1.0

- Seat-specific booking with unique seat constraints instead of aggregate inventory.
- Waiting room / virtual queue for extreme flash-sale spikes.
- Split hot inventory into section/seat buckets to reduce lock contention.
- Read replicas for catalog/search traffic.
- OpenTelemetry distributed tracing after services are actually split.
- Kubernetes and autoscaling only if the deployment objective specifically requires it.